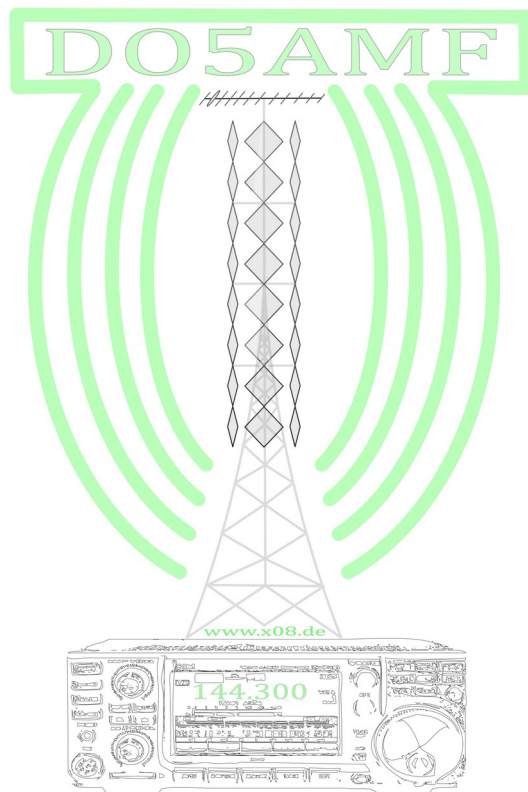

Chat-Mining: Jagdinstinkte wecken mit KST4Contest

Inhaltsverzeichnis

1) Vorwort.....	3
2) Chat – Mining – was soll das sein?.....	4
3) Definitionssache(n).....	5
a) Aufenthaltsort.....	5
b) Aufenthaltsdauer.....	5
c) Erfolg und Misserfolg (im Chat).....	5
d) Chat-Effizienz.....	6
4) Interessante Zahlen an einem Beispiel.....	7
a) Anzahl der Nachrichten.....	7
b) Teilnehmerzahlen.....	8
c) Sender vs. Empfänger.....	9
d) Zeitliche Aktivitätszentren.....	9
e) Big guns im Chat.....	10
f) Erfolgsquote(n).....	11
1) Zusammenhang von Contest-Score und Chateinsatz.....	11
2) Zusammenhang von Contest-Score und „positive signals“.....	13
3) Zusammenhang von Contest-Score und Sked-inits.....	14
4) Erfolgsrezept.....	15
5) KST4Contest v1.31 – Update.....	17
a) Entwicklung eines KST-Simulators.....	18
b) Win-Test support.....	18
c) Zweite Chatkategorie.....	19
d) QSO-Sniffer.....	19
e) Smart Frequency-recognition.....	20
f) Priority-Score und seine Folgen.....	21
g) Skeds.....	23
h) Aircraft-Scatter timeline.....	24
i) PSTRotator-Interface.....	25
j) Auto-Answer.....	25
k) Dark Mode.....	26
l) DXCluster Server, AP Info Integration.....	27
6) Integration der Software in den Contestablauf.....	27
7) To-do-Liste.....	29
8) ZDF - Zahlen, Daten, Fakten.....	30
a) Entwicklungszeit und Aufwand.....	30
b) Danke.....	30
c) Umfang.....	30
d) Reich wird man damit nicht.....	30
9) Download.....	31

Chat-Mining: Jagdinstinkte wecken mit KST4Contest



Chat-Mining: Jagdinstinkte wecken mit KST4Contest

1) Vorwort

Hinweis zur Einordnung: Dieses Dokument gibt den Entwicklungs- und Kenntnisstand zur GHZ-Tagung Dorsten 2026 wieder. 2024 wurde OM Ralf, DG6EA, auf meine Aktivitäten bezüglich des ON4KST-Chats und der Stromversorgung unserer Conteststation DM5M aufmerksam, und wir wüfelten das Thema ON4KST-Chatclient aus. Ich durfte auf der GHZ-Tagung in Dorsten 2025 darüber sprechen. Das Thema wurde überraschend gut aufgenommen und intensiv diskutiert. Im Hinblick auf die Ideen und die Roadmap für das Programm, die ich 2025 in Aussicht stellte, gab es manches Raunen im Saal. Das ließ die Weiterentwicklung bestimmter Funktionen von vornherein begrüßenswert erscheinen, schloss andere in Teilen aber auch aus.

Leicht entrüstetes Gemurmel betrat den Raum, als ich erwähnte, dass Stationen im Chat automatisiert angeschrieben werden könnten, sofern man sie noch nicht gearbeitet hatte. Schon nach diesem Ereignis beschloss ich, dass ich dieses Feature wieder von der Liste streichen werde. Ich darf alle beruhigen: dabei ist es auch geblieben.

Hiermit stelle ich ein Skript zu meinem Vortrag in Dorsten im Jahr **2026** bereit. Das Skript aus dem letzten Jahr ist im Tagungsband der GHZ-Tagung verfügbar und bietet einige Erläuterungen für Designentscheidungen und Hintergründe, auf die ich in diesem neuen Vortrag aufbaue und auch aufbauen muss, weil eine Gesamtübersicht den Rahmen des Skripts sprengen würde.

Ich hoffe, der geneigte Leser hat noch nicht alles vergessen, was im ersten Teil meines Vortrages thematisiert wurde – ansonsten darf es gern nochmals nachgelesen werden, Ralf hat bestimmt noch Exemplare aus dem letzten Jahr vorrätig.

Der Vortragstitel bereitete mir diesmal schlaflose Nächte, weil ich anfangs nicht genau wusste, auf was er hinauslaufen würde und mit welchen Ergebnissen in der konkreten Implementierung gerechnet werden konnte. Die Implementierung stützte sich hauptsächlich auf theoretische Überlegungen, die die Chat-Auswertung mit sich brachte. Daraus konkrete Implementierungen zu machen, ist zum Teil komplex und wird von mir dauernd weiter verfolgt. Zum Zeitpunkt des Vortrages wird der Funktionsumfang und das konkrete Aussehen der Anwendung bereits wieder anders aussehen, als hier, da das Skript ca. 4 Wochen vor der Tagung fertig sein sollte.

Viel Spaß beim Nachlesen und auf der für mich zweiten GHZ-Tagung 2026 in Dorsten es

73 de DO5AMF, Marc

2) Chat – Mining – was soll das sein?

Data Mining klingt sperrig, jeder hat aber schon einmal irgendwo davon gehört. Definitionsgemäß bedeutet es: automatisierte Analyse großer Datenmengen zur Findung von Mustern, Korrelationen und nützlichen Informationen. Ziel in der IT ist, damit Prozesse zu optimieren. Das ist ja, wie wir aus dem letzten Jahr wissen, das Hauptziel des Chatclients KST4Contest.

Dass Meta- und Kontextinformationen aus dem ON4KST-Chat eine wertvolle Informationsquelle sein können, habe ich letztes Jahr bereits erläutert und in der Vorgängerversion von KST4Contest auch bereits genutzt. Es gibt einige Informationen, die wir uns direkt zu Nutze machen können, um zur Laufzeit des Chatclients Mehrwert daraus zu ziehen. Dazu zählen ganz unmittelbar die Textinhalte, aus denen wir zum Beispiel eine **Frequenzangabe** extrahieren können.

Darüber hinaus gibt es einige Informationen, die mittelbar Mehrwert bieten, weil wir daraus Informationen ableiten können, die dem geneigten Nutzer des Chats nicht augenfällig sind. Dazu zählt zum Beispiel das **zeitliche Betriebsintervall** einer bestimmten Contest-Station.

Zuletzt gibt es noch Informationen, die interessant sind, weil sich sonst noch niemand damit beschäftigt hat. Vielleicht bieten auch diese irgendeinen Mehrwert...wenn nicht, sind sie immerhin unterhaltsam zu lesen. Wenn sich größere Datenmengen ansammeln, bedingt mein Informatiker-Dasein, dass ich mich damit befasse. Im KST Chat reden wir von 130.000 Zeilen Text und mehr. Da muss sich doch noch etwas herausholen lassen.

Man braucht eine solide Datenquelle. Die liegt uns zum Glück offen vor. Die Chatlogs sind auf der Hauptseite von www.ON4KST.org immerhin ein gutes Jahr lang abrufbar; Unter „Menu“ → „chat review“ kann man nach der Auswahl eines Datums den gesamten Chatverlauf als Text abrufen. Man kann diesen Text auch filtern, zum Beispiel nach einem Rufzeichen oder einem Text-Pattern. Das habe ich nicht getan, da ich die gesamte Datenbasis für die Auswertungen brauchte.

Ich habe die Chatlogs zu IARU Contesten aus den letzten 2 Jahren ausgewertet.

Unter anderem auch das für den Juli-Contest 2025. Exemplarisch zeige ich an diesem ermittelbare Informationen. Die Schlussfolgerungen fielen für alle Contests sehr ähnlich aus, was die Verhältnisse der Zahlen angeht. Die absoluten Zahlen unterscheiden sich zum Teil stark, die Tendenzen ähneln sich aber.

Ich habe bewusst den Juli-Contest 2025 als Beispiel zum Zeigen konkreter Zahlen gewählt, weil ich hier die Daten von zwei Chatkategorien (144/432 und Microwave) gemeinsam als Basis nehmen und Korrelationen auch zwischen den unterschiedlichen Kategorien suchen kann.

3) Definitionssache(n)

Um Schlussfolgerungen aus den Daten ziehen zu können, muss man sich überlegen, wonach man überhaupt sucht. Wie sagte schon mein Prof. Dr. Wolfgang Schramm (HS Mannheim) in Algorithmen und Datenstrukturen: „am Anfang steht das weiße Blatt“. Allerdings bin ich das ja schon gewohnt und so definierte ich einige Parameter zur Erstellung einer Analyse, die da wären...

a) Aufenthaltsort

In welchem Chat war ein Rufzeichen? Hat es den Chatkanal gewechselt? Wie oft? Wo schreibt es?

b) Aufenthaltsdauer

Wie lange war das Rufzeichen insgesamt da? Wie lange sind die Stationen überhaupt im Chat erreichbar (Zeitintervalle)? Kann man daraus auf Betriebszeiten schließen?

c) Erfolg und Misserfolg (im Chat)

Es gibt eindeutige Schlüsselphrasen, mit denen man auch ohne das Vorhandensein eines QSO-Logs der jeweiligen Teilnehmer eines Contests feststellen kann, dass ein Sked erfolgreich war. Beispiele sind „tnx, 73“, „tnx qso“, „tnx for xxx points“ etc. Ebenso gibt es eindeutige Schlüsselphrasen, die belegen, dass ein Sked nicht erfolgreich war. Beispiele sind „tnx try“, „no copy“, „no ping, sry“.

Ich versuche zunächst, Erfolg oder Misserfolg eines Skedversuchs zu messen, ohne explizit gesicherte Angaben zu Absicht, Anfang und Ende auf den Sked einer Station zu haben. Das ist typischerweise kaum möglich, da niemand Interesse daran hat, extra Dokumentationsarbeit für Statistiken zu verrichten. Daher begann ich mit der Methode durch Textauswertung:

- **Sked-phrase** auswerten (Sked-**Absicht** erkennen, z.B. „try sked 144.210?“)
- **Sked-Zeitfenster** starten (Ich werde das später „**init**“ oder „**window**“ nennen)
- **Sked-phrase** auswerten (Sked-**Ergebnis** erkennen), Sked-Zeitfenster stoppen oder terminieren (falls keine Stop-Nachricht gefunden werden kann)

In einigen Fällen führt das zum Erfolg. In den Allermeisten Fällen **nicht**! Das liegt zum einen daran, dass ich den Erfolg nur sehr rudimentär aus Chatphrasen ableiten kann, da ein Großteil der Kontakte naturgemäß über den Äther bestätigt und abgewickelt wird. Es kommt noch dicker: es gibt erheblich(!) mehr Anfragen, als Antworten. Schon allein deshalb, weil es eine Masse an Stationen gibt, die periodisch Stationen anschreibt und darauf entweder direkt gefunkt wird (empirisch bei uns ermittelt: das ist meistens der Fall), oder aber – das ist mit Abstand häufiger – auf diese Anfrage keine Antwort erhält. Gründe dafür könnten sein, dass der Angeschriebene keine Zeit oder kein Interesse hat. Vielleicht nutzt er auch schon KST4Contest und benötigt daher keine

Chat-Mining: Jagdinstinkte wecken mit KST4Contest

periodischen Einladungen auf die QRG anderer Stationen oder hat es schon fünfmal erfolglos versucht und ignoriert die Anfragen ab dann...

Im späteren Verlauf der Text-Analysen versuche ich auch herauszufinden, ob es sinnvoll ist, **Massen-Mailings** zu versenden, also gleiche Nachrichten an eine Liste von Stationen. Anders gefragt: sind „Spammer“ erfolgreicher, als Stationen, die sparsamer und gezielter mit ihren Texten umgehen? (Stichwort: Qualität > Quantität?). Zum Zeitpunkt der Erstellung dieser Zeilen ist das Ergebnis noch nicht bekannt und ich hoffe, dass ich ein Ergebnis dazu bekomme.

Intuitiv führt das Einladen nicht gearbeiteter Stationen auf die eigene QRG zu guten Erfolgen. Im Falle DM5M funktioniert das besonders häufig mit Stationen aus 9A und G. Die antworten dann nicht im Chat, sondern kommen auf die genannte QRG und versuchen es. Anders herum erlebe ich als DO5AMF/p mit 3ele und 10W an der Ostsee, Stationen wie OE1W. Innerhalb einer Stunde kamen 10 Sked-Anfragen mit ihrer QRG, die ersten lehnte ich dankend ab. Sie kamen weiterhin. Ich war ca. 6 Stunden qrv und erhielt die meisten überflüssigen Nachrichten von OE1W.

Es erscheint sehr sinnvoll, andere Stationen insbesondere dann anzuschreiben, wenn die Antennen wirklich schon in ihre Richtung stehen und der **Pfad funktionieren kann**. Exzessives Anschreiben ohne Berücksichtigung der Pfad-Bedingungen oder gar der Antennenrichtung wird schnell dazu führen, dass solche Stationen nach einigen Fehlversuchen nachhaltig ignoriert werden, selbst, wenn sich z.B. AS-Bedingungen verbessern und der Pfad möglich wird.

d) Chat-Effizienz

Schon in meinem letzten Vortrag habe ich das Thema angeschnitten. Über Sinn und Unsinn der Verwendung des Chats für UKW kann man diskutieren. Gern wird er auch verteufelt. Vielleicht gieße ich Öl ins Feuer, vielleicht schaffe ich aber auch das Gegenteil, wenn ich versuche, die Auswirkungen der Chatanwendung sichtbar zu machen. Die Möglichkeiten über die Messung der „**positive Signals**“ (Antworten wie „tnx qso“) sind schnell ausgeschöpft und bieten wenig Aussagekraft, weil sie insgesamt zu selten vorkommen.

Daher wird der Weg sein, Korrelationen zwischen dem Scoreboard und dem Chat zu suchen, um eine Aussage über die Effizienz des Chateinsatzes treffen zu können und Optimierungspotenziale abzuleiten.

Genug des trockenen Einleitungsmaterials; nun liefere ich ein paar interessante Daten, die sich aus diesen Analysen ergeben haben und hin und wieder auch Schlussfolgerungen auf die eigene Betriebstechnik bzw. direkten Einfluss auf den Chatclient KST4Contest, ehe ich dann die Neuerungen vorstelle.

4) Interessante Zahlen an einem Beispiel

Ich erwähne hier ausdrücklich, dass ich hier das **Beispiel IARU UKW+ 07/2025** zeige. Die Ergebnisse der unten genannten Zahlen und Statistiken sind für alle anderen IARU UKW Contests sehr ähnlich, sodass sich auch allgemeine Aussagen treffen lassen.

a) Anzahl der Nachrichten

Im Juli Contest 2025, der auf UKW stattfindet und frequenztechnisch nach oben fast keine Grenzen kennt, gab es insgesamt **117.399** Nachrichten, davon **111.206** gerichtet und **6.139** ungerichtet (also ohne expliziten Empfänger, ich nenne das „CQ“ bzw. „broadcast“). Das Verhältnis von gerichteten zu ungerichteten Nachrichten tendiert im microwave Chat deutlich mehr zu adressierten Nachrichten. 97% der Nachrichten sind adressiert, im UKW Chat sind es nur 93%.

Damit unterscheidet es sich stark vom September-Contest 2025, in dem es deutlich mehr Broadcasts waren. Die Masse der Broadcasts erklärte sich zu großen Teilen aus der fehlerhaften Syntax von Nachrichten, die als PM gemeint waren (siehe Skript 2025). Man erkennt diese, weil sie Phrasen wie „CQ <Callsign>“ (ohne „/“) oder „(Callsign) <message>“ enthalten.

Ein großer Prozentsatz der Chatclients heben diese Nachrichten dann nicht hervor, was wahrscheinlich dazu führt, dass die Nachricht vom Empfänger nicht gelesen wird. Der Anteil solcher Nachrichten lag im September bei ca. 8 % (das Muster zum September 2024 ist fast gleich).

Interessant: In NAC und UKAC Contests liegt der Prozentsatz der „Broadcasts, die keine sein sollten“, ca. doppelt so hoch.

Im letzten Jahr hatte ich das Problem bereits adressiert und in KST4Contest auch gelöst, indem ich **solche Nachrichten als PM** verarbeite.

Chat-Mining: Jagdinstinkte wecken mit KST4Contest

b) Teilnehmerzahlen

Es gab insgesamt **1172** eindeutige Rufzeichen im Chat in den 24h des Contests (UKW + mw).

Es gab insgesamt **964** eindeutige Rufzeichen nur in Kategorie 2 (144/432).

Es gab insgesamt **411** eindeutige Rufzeichen nur in Kategorie 3 (microwave).

In Kategorie 2 (aber nicht 3) gab es **765** Rufzeichen (als Sender und Empfänger!).

In Kategorie 3 (aber nicht 2) gab es **210** Rufzeichen (als Sender und Empfänger!).

Zwischen den Kategorien gewechselt haben **144** (nur Sender „messbar“).

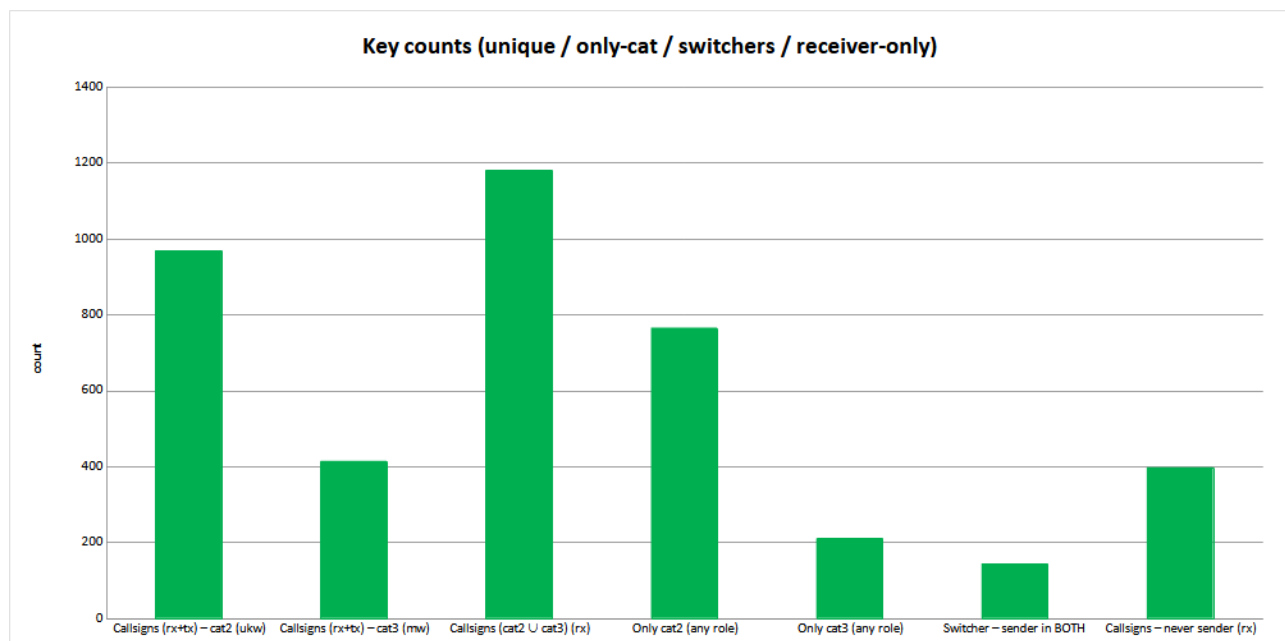


Abbildung 4.1: Unique Callsigns in den einzelnen Kategorien, nach rx/tx und Kategorie

Damit hätten wir eine Tendenz der Antwort auf die Frage, ob es sinnvoll wäre, beide Kategorien zu beobachten. Sobald man im Mischbetrieb UKW + 23cm (oder höher) funkt (oder anders herum), lautet die Antwort ganz klar: „ja“.

Die Stationen, die auf 23cm und höher funken, machen sich nicht immer die Mühe, den 144/432 MHz Chat zu betreten oder zu beobachten. Das macht Sinn. Zumindest formal. Solche Teilnehmer haben keine Gelegenheit, Sked Partner (wie uns als DM5M) zu gewinnen, die das 23cm als „Eh da“-Band stiefmütterlich mit aktivieren, weil der IC9700 es eingebaut hat und wir das nicht einfach brach liegen lassen möchten.

Zum Glück ist das ein Thema, welchem man mit Software beikommen kann. In 03/2025 fügte ich in KST4Contest die **Nutzung einer zweiten Kategorie mit Single Sign-on** hinzu. KST4Contest ist übrigens nicht der einzige Client, der das kann. Der Logger VUSC4WIN kann es auch. Dieser Logger ermöglicht übrigens auch die heißgeliebten **Mass-Mailings...**

Chat-Mining: Jagdinstinkte wecken mit KST4Contest

c) Sender vs. Empfänger

Niemals gesendet, also ausschließlich mitgelesen, haben insgesamt **397** Rufzeichen. **355** Rufzeichen haben in Kategorie 2 nur empfangen, **94** in Kategorie 3 nur empfangen.

Dass $355 + 94$ nicht 397 ergibt, ist kein Widerspruch. Es begründet sich darin, dass einige Callsigns der jeweiligen Teilmengen in einer Kategorie gesendet, in der Anderen aber nur empfangen haben. Die 397 Rufzeichen beziehen sich auf Rufzeichen, die wirklich überall nur als Empfänger auftraten.

Schlussfolgerung: Es gibt viele Stationen, die nur in einer der beiden Kategorien antworten und: da diese Stationen nicht 24/7, 365 Tage im Jahr im Chat sind (das sind nur etwa 40), müssen sie ein Interesse daran haben, Nachrichten zu empfangen – selbst dann, wenn sie nicht antworten. Es lohnt sich scheinbar auch, Calls anzuschreiben, die nicht antworten (manche dürfen in bestimmten Contests nach Contestregeln nicht antworten, wie mir Tim, **DK5OH** früher schilderte).

d) Zeitliche Aktivitätszentren

Der Peak des Nachrichtenaufkommens ist am Sonntag ab 7:00 Uhr UTC. Das ist für alle von mir ausgewerteten IARU Contests ähnlich. Hieraus kann abgeleitet werden, wann der beste Zeitpunkt zur Teilnahme für die 6 Stunden Klasse ist, **wenn man stark auf Skeds setzt oder auf Skeds angewiesen ist**. OMs wie **SM6VTZ** und **SK7EYW** sind in der 6-h Klasse oft in diesem Zeitraum qrv. Neben Facebook ist eine gute Quelle für solche Teilnahme-Informationen diese hier: <https://qrvmmap.vhfdx.ro/>

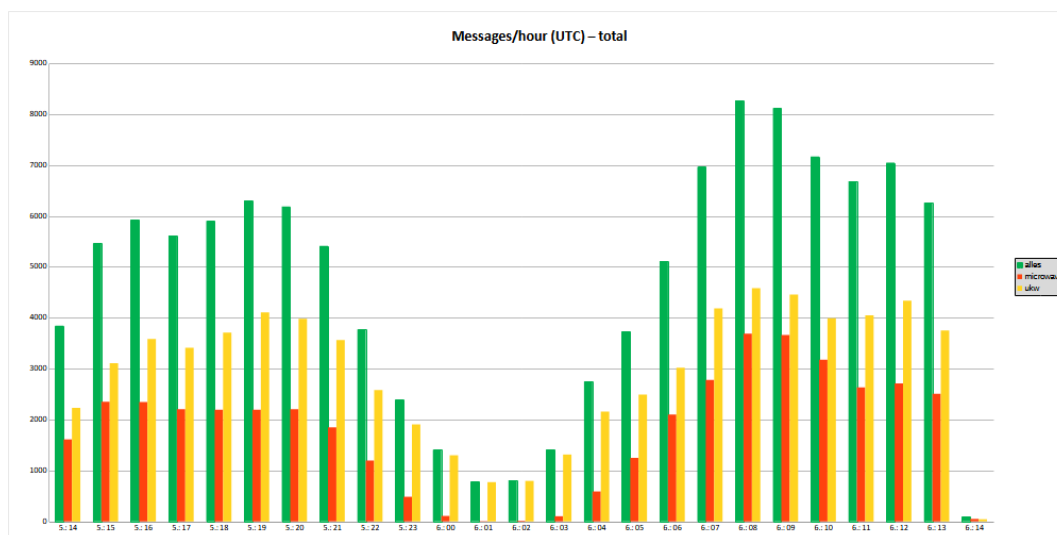


Abbildung 4.2: Nachrichten pro Stunde in den Chat-Channels

Diese Erkenntnis ist etwas **trügerisch**, deswegen schreibe ich das explizit:

Das Zentrum der **Aktivität im Contest** liegt typisch ganz klar am Anfang des Contests. Das geht schon aus typischen QSO-Raten der ersten Stunde hervor.

Das Zentrum der **Aktivität im Chat** liegt am Sonntag gegen 7-8 Uhr UTC!

Chat-Mining: Jagdinstinkte wecken mit KST4Contest

Man könnte mutmaßen, dass am Samstag genügend Bandbelegung dafür sorgt, dass Skeds nicht unbedingt notwendig sind. Es werden zuerst die QSOs mit großen Signalstärken durchgeführt. Die schwierigen QSOs, die Vorarbeit benötigen und nicht automatisch durch CQ-Rufen und Band-Absuchen gelingen, werden auf die Nacht oder Sonntag terminiert.

Schwache Stationen und Stationen außerhalb des Zentrums Europas sind auf Skeds angewiesen. Deswegen legen sie die Aktivität vermutlich auf die Sonntage. Aus der Praxis: als ich selbst in SM an einem UKW-Contest (Juli 2020) teilnahm, drehte die Antenne fast niemand versehentlich nach Norden. Von befreundeten Oms in SM und LZ habe ich ähnliche Rückmeldungen erhalten.

Aus diesem Grund kamen auch explizit von diesen Oms wiederholt die Nachfragen eines QSO-Spions, der es ermöglicht, gezielt die Nachrichten lokal benachbarter Stationen mitlesen zu können, um die Gelegenheiten nutzen zu können, wenn potenzielle Gegenstationen die Antennen für Skeds in ihre Richtungen stellen. Aus diesem Grund habe ich im Dezember 2025 den **QSO-Sniffer** implementiert. Er ermöglicht es, gezielt die Nachrichten an bestimmte Callsigns mitzulesen.

Das „**Sked directed to me**“-Feature habe ich schon wesentlich früher implementiert. Der Zweck geht in eine ganz ähnliche Richtung, beinhaltet aber mehr Heuristik und benötigt weniger Aufmerksamkeit des Operators. Die Rückmeldungen dazu sind durchweg sehr positiv!

e) Big guns im Chat

Über die Frequenzen kann man sich mit einigem Materialaufwand Gehör verschaffen. Eine gängige Idee ist, neben der Anzahl der Antennen auch die der Antennenrichtungen zu steigern und natürlich kann man auch Leistung hinzufügen. In Kombination wird eine solche Station dann auffallen und als „Big Gun“ bezeichnet. Auch im Chat kann man auffallen. Der Mitteleinsatz ist hier überschaubar.

Der ON4KST Chat selbst stellt keine Board-Mittel zur Messung der Sichtbarkeit von Teilnehmern bereit, deswegen habe ich mir das Thema genauer angesehen.

Chat-Mining: Jagdinstinkte wecken mit KST4Contest

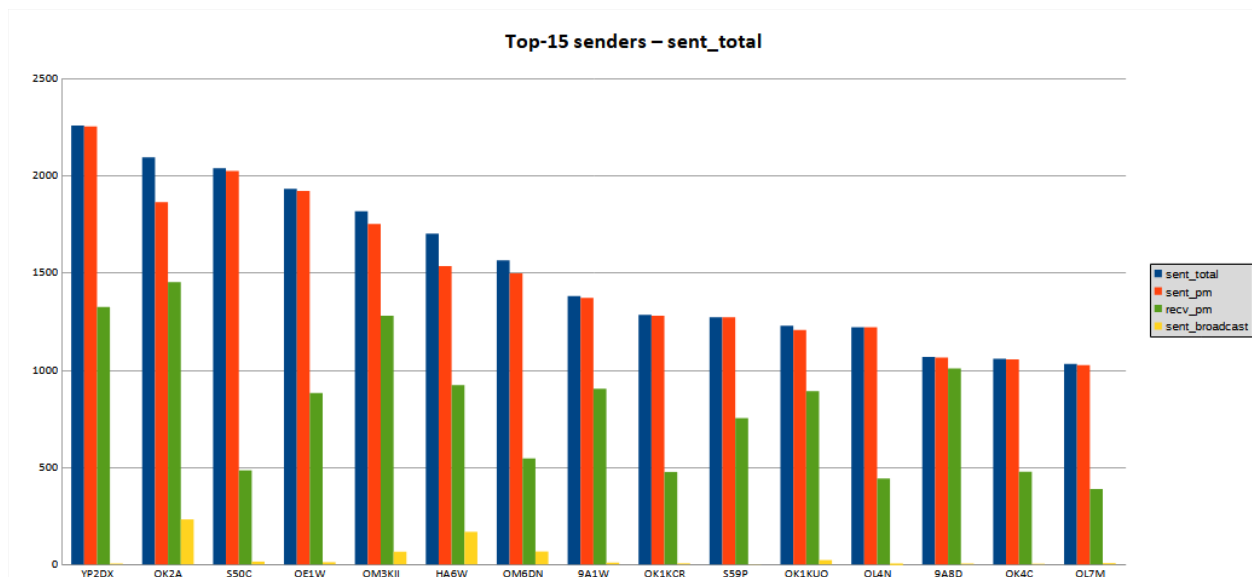


Abbildung 4.3: Top 15 Nachrichtenversender und Antwortverhalten auf ihre PM

Weitere Auswertungen ergaben, dass **S50C** im Schnitt die meisten Nachrichten pro Stunde versendet hat. Der OM an den Tasten hatte also Durchhaltevermögen! **OE1W** folgte direkt darauf, aber mit gut 30% weniger Nachrichten (im Schnitt). Im Diagramm ist nicht die Nachrichten/Stunde-Charakteristik zu sehen, sondern ausschließlich die absoluten Zahlen der Top 15 unter den Nachrichtenversendern. Abgebildet ist auch, wie viele PM im Verhältnis empfangen wurden, was ein Gefühl dafür aufkommen lässt, ob daraus Skeds entstanden sind.

f) Erfolgsquote(n)

1) Zusammenhang von Contest-Score und Chateinsatz

Was mich seit einiger Zeit zu wissen reizt, ist, genauer zu erfahren, wie sich der Einsatz des Chats auf die Ergebnisse im Contest auswirken könnte. Der erste Versuch war, die Frage zu stellen: „Wie viele Stationen pro Wertungsklasse in Top 20 Platzierungen gibt es, die den Chat nicht genutzt haben?“. Eine Antwort gibt es auch. Aber das würde zu kurz greifen.

Für eine sinnvolle Betrachtung muss auch berücksichtigt werden, wie viele Teilnehmer es pro Klasse gab (wenn es nur 10 wären, wären ja alle in den Top 20!). Es bietet sich also an, nicht fest Platz 1-20 auf Chatnutzung zu analysieren, sondern die Anzahl der TOP-Platzierungen auf die Anzahl der Chatnutzer zu normalisieren.

Verschweigt man dem Leser diese Information der Normalisierung, so sind die Ergebnisse irreführend und erfüllen nicht den Anspruch einer neutralen Betrachtung. Das habe ich im Studium der Informatik vor 15 Jahren gelernt und wieder vergessen. Jetzt tauchen diese Begriffe wieder auf.

Chat-Mining: Jagdinstinkte wecken mit KST4Contest

Grüße gehen heraus an Prof. Dr. Frank Nürnberg: es war nicht alles vergebens.

Ich stelle jetzt bewusst 3 Diagramme untereinander dar und biete danach eine Schlussfolgerung an.

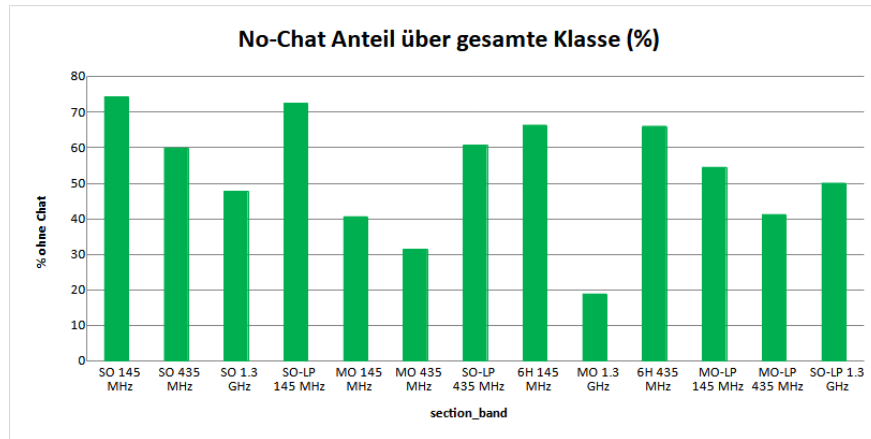


Abbildung 4.4: Wertungslogs, pro Klasse ohne Chateinsatz

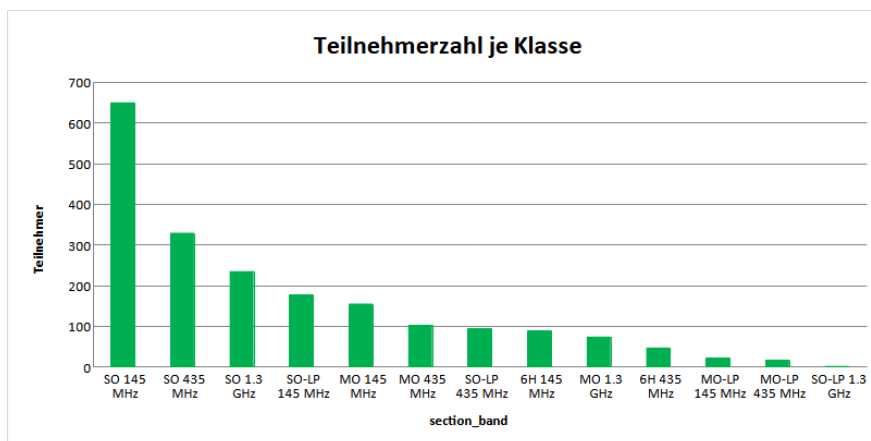


Abbildung 4.5: Wertungslogs, pro Klasse gesamt

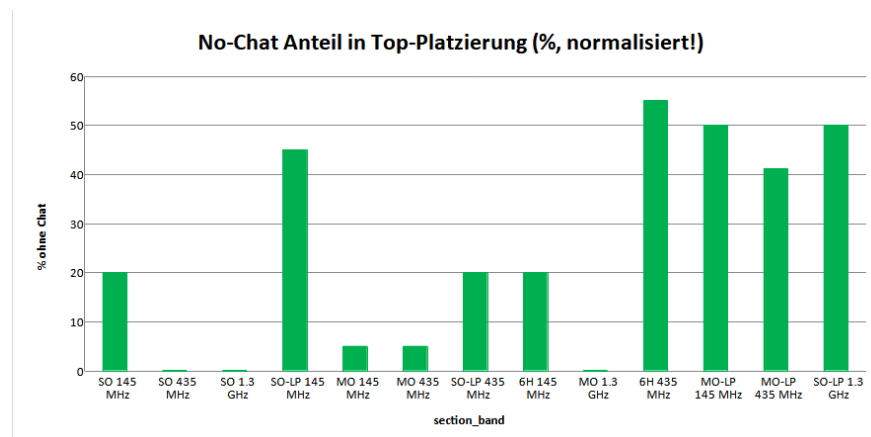


Abbildung 4.6: Wertungslogs pro Klasse ohne Chat auf Top20 Platz

Chat-Mining: Jagdinstinkte wecken mit KST4Contest

Wenn ich aus den drei Diagrammen zusammen einen Rückschluss ziehen müsste, wäre es dieser:
„Der KST-Chat bildet die Gesamtteilnehmer einer Wertungsklasse nur unvollständig ab – aber in den großen Klassen ist Chat-Nutzung bei den Top-Stationen deutlich überrepräsentiert.“

Der Grund für das umständliche Herantasten an das Thema „Erfolg durch Chateinsatz“ ist, dass mir außer unseren eigenen Wertungslogs keine Log-Daten vorliegen. Ich kann sie nicht einfach mit potenziell ermittelten Sked-Zeitfenstern der Top-Stationen der IARU-Rangliste zusammenbringen, um eine Tendenz für QSOS mit oder ohne Chat-Einsatz zu ermitteln.

Bei DM5M ist das Ergebnis dahingehend ganz eindeutig. Besonders im Reichweiten-Grenzbereich unserer Anlage kann nachgewiesen werden, dass Stationen kurz nach dem Sked-Angebot (auch ohne positive Rückmeldung via Chat) gefunkt wurden und auch, dass das zum Teil explizit über airplane scatter bewerkstelligt wurde. In der Log-Analyse kam ich über mehrere Logs auf etwa 8% der QSOS. Ein überraschend hoher Wert. Man kann daraus aber nicht folgern, dass diese QSOS ohne Chat unmöglich gewesen wären, auch, wenn sie in diesen Fällen explizit aus Sked-Anfragen entstanden!

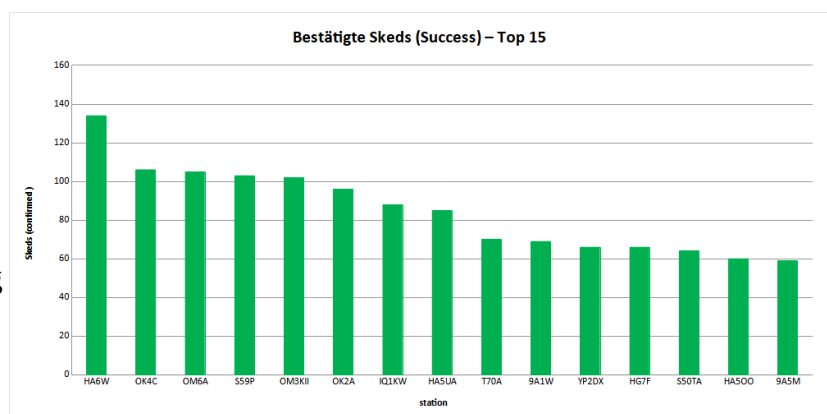
Unsere eigene Stichprobe ist zu klein, um Allgemeinaussagen treffen zu können. Deswegen versuche ich, andere Erfolgsfaktoren zu messen, als die IARU Scores, in der Hoffnung, trotzdem einen Zusammenhang dort hin zu finden.

2) Zusammenhang von Contest-Score und „positive signals“

Wie wir den Sked-Erfolg innerhalb des Chats definieren und ermitteln könnten, beschreibe ich in (3)c) Erfolg und Misserfolg (im Chat)). Wir bewerten „positive Signale“ in Antworten auf gesendete Sked-Init-Nachrichten.

Hier kann man auch eine Topliste ermitteln. Auffallend ist, dass Nachrichten mit „positive signals“ unterrepräsentiert sind. Korrelation mit der Rangliste: die Rufzeichen sind zwar alle bekannt, es gibt aber keinen erkennbaren Zusammenhang zum Platz im Scoreboard.

Meine Grundannahme ist: es gibt einen Zusammenhang zwischen Scoreboard und Chat-Verhalten. Eine Sichtbarkeit dieser Annahme kann man nicht erzwingen. Wenn man diese Korrelation nicht sieht, ist die Stichprobe vielleicht falsch gewählt oder die Grundannahme falsch. Im nächsten Schritt wurden nicht die Top 15, Top 30 oder ähnliches auf



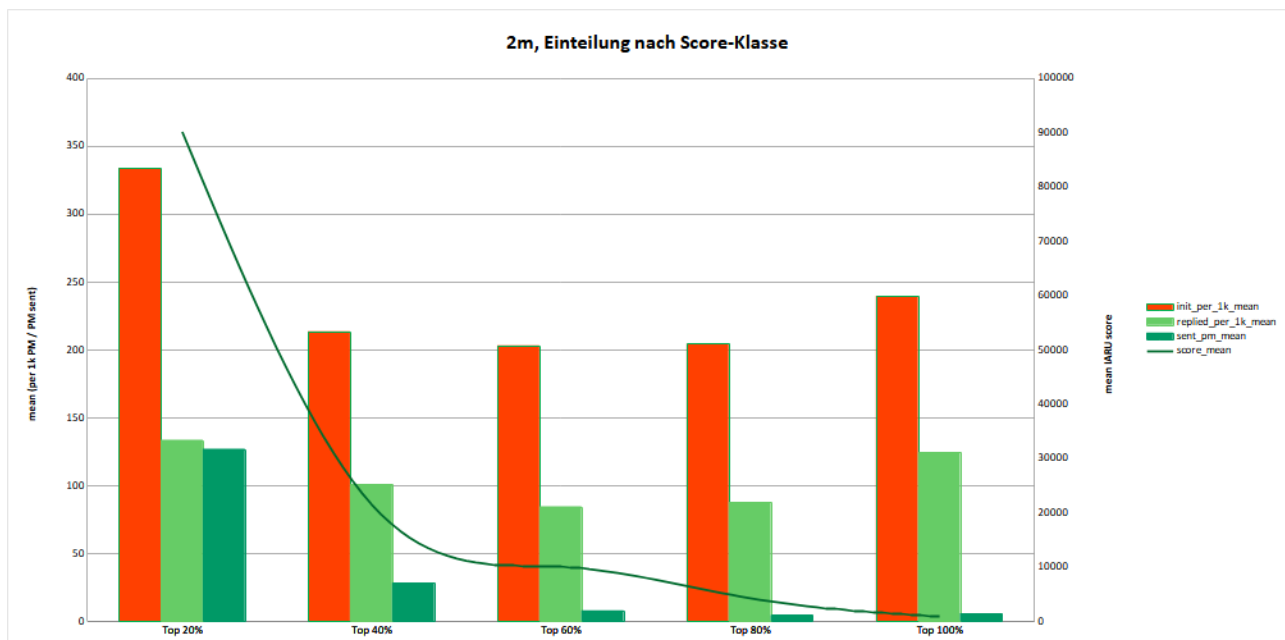
Chat-Mining: Jagdinstinkte wecken mit KST4Contest

Korrelationen zum Scoreboard betrachtet, sondern die besten 20% der Topliste, die nächsten 20%, etc.

Das führt zu 5 „bin“-Klassen, die jeweils 20% des Scoreboards einer Wertungsklasse repräsentieren und deren Verhalten bzgl. Nachrichtenversand man mit dem Scoreboard zusammen in Relation setzen könnte.

3) Zusammenhang von Contest-Score und Sked-inits

Klingt sperrig, ist es aber nicht. Im Juli-Contest führt das Einteilen in Ergebnisklassen zu folgendem Bild. Ich bilde hier nur VHF Multi Operator ab. Die anderen Wertungsklassen (Multi) sehen ähnlich aus. Die Single Klassen nicht immer, vor allem für die low power Klassen ist der Trend nicht so stark (aber auch) sichtbar.



Die Stationen mit den größten Korrelationen zum Scoreboard sind die, die viele **Skeds** anstoßen **und** daraus viele **Sked-Zeitfenster** generieren – also **Antworten durch eine Sked-Anforderung** erzeugen. Die relative bzw. peak/Stunde-Anzahl versendeter PM ist ein Indikator, der damit zusammenhängt: „Wie man in den Wald hineinruft.“. Die Erzeugung von Sked-Fenstern (also die Tendenz für Antworten), sieht wie ein Hebel aus.

Ich biete wieder eine Schlussfolgerung an:

Die Qualität der Anfragen von Sked-Erzeugern könnte ein Indikator dafür sein, mehr QSOs zu bekommen.

Anders ausgedrückt: mehr direkte Nachrichten zu schreiben, wirkt sich zwar aus. Deutlicher ist der Trend aber, wenn durch mehr kontaktfähige, verwertbare Sked-Requests Antworten erzeugt werden (hier spielt auch das Intervall eine Rolle, je kürzer, desto besser). Ganz ähnlich zum Marktstand:

Chat-Mining: Jagdinstinkte wecken mit KST4Contest

laut rufen ist nicht verkehrt, das lockt Kunden an. Es ist aber noch besser, wenn das Angebot erstens sichtbar und zweitens attraktiv ist. Attraktives Marktpersonal spielt auf Band keine Rolle, der Vergleich hinkt also doch...

Da wir nun einen Indikator dafür haben, welche Stationen tendenziell mehr Sked-Windows erzeugen, können wir uns fragen, woran das liegt. Eine Textanalyse, wie sich die Texte von Top 20% Stationen von anderen unterscheiden, zeigte, dass Stationen mit hoher Reply-Rate nicht zwingend „vollständige“ Erstnachrichten senden. Sie bekommen Replies, weil sie die richtigen **Stationen zur richtigen Zeit** anschreiben und weil es inhaltliche Unterschiede gibt. Das bedeutet aber nicht, dass die Vollständigkeit einem Sked-Request schadet.

4) Erfolgsrezept

Die Auswertungen ergaben ein starkes Muster. Mehr sofort verwertbare Sked-Infos führen zu mehr Replies.

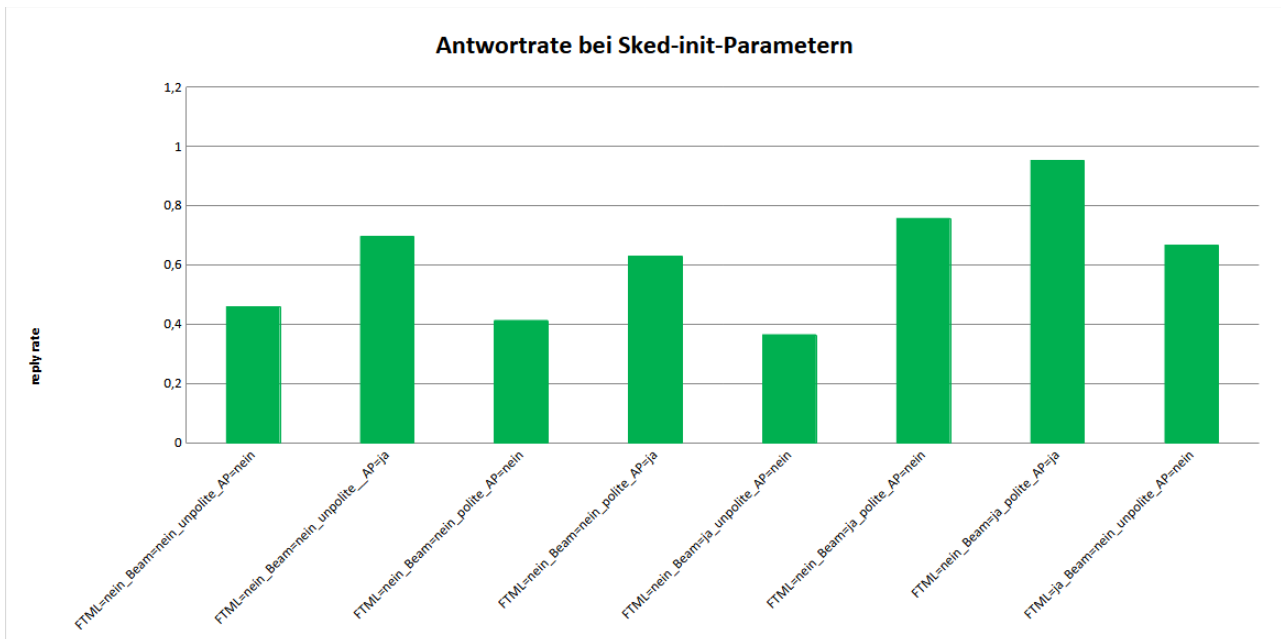
Genauer betrachtet ist das kein überraschendes Ergebnis. Es ist erfreulich, dass es diese Korrelationen gibt, da der Workflow durch deren Kenntnis optimiert werden kann.

Wie im Diagramm bereits erkennbar ist, führt ein optimierter Inhalt dazu, dass Sked-Anfragen besser angenommen werden. Gut reflektierte Sked-Anfragen können enthalten:

- Vorhandensein von Action-/Sked-Wörtern (qrg/qrv/ant/dir/call/beam/sked/ap...)
- Frequenzangaben in Messages („F“)
- Timing-Angaben (Uhrzeit oder Laufzeit bis start) („T“)
- Höflichkeits-/Ankerwörter (pse/tnx/hi/...) („POL“)
- Beam-Richtungshinweis („BEAM“)
- Locator („L“) – wobei der keine große Rolle spielt
- Airplane („AP“) – Hinweis
- Mode („M“) – spielt auch kaum eine Rolle

Erstellt man daraus ein Diagramm, dann wird erkennbar, wie man am meisten Antworten provoziert.

Chat-Mining: Jagdinstinkte wecken mit KST4Contest



Ob freundlich oder nicht: AP-Angaben und Antennenrichtungshinweis bringen messbar bessere Erfolge, selbst wenn die FTML-Angaben nicht vollständig sind oder nachgereicht werden müssen bzw. vorangegangen sind.

Die Informationen müssen für die Öffnung eines Sked-Initiierungen nicht in der ersten Anfrage vorhanden sein. Es reicht, wenn sie **zeitnah** nacheinander eintreffen, was auch häufig der Fall ist. Schlüsselst man die FTML-Angaben nicht so fein auf, so ist der Trend extrem gut erkennbar. Sked-Initiierungen mit AP und Beam-Daten führen zu fast 30% höheren Antwortquoten als z.B. „Hi noname, try 2m?“.

Sie zeugen auch davon, dass bereits Vorbereitungszeit in den Sked geflossen ist. Auf höflich oder personalisiert ausgeführte Anfragen wird tendenziell mehr geantwortet, als auf andere.

Die Gesamtzahl privater Nachrichten zu erhöhen, führt fast automatisch auch zu einer höheren Reply-Rate aber der Zusammenhang ist nicht stark. Sehr gut sichtbar ist das in den letzten Contesten immer wieder an S50C und OE1W gewesen, die sich bzgl. Rang 1 beim Nachrichtenversand immer wieder abwechseln.

Zusammenfassend würde ich sagen, der Spam lohnt sich nicht. Insbesondere den Eindruck, der mir bei der letzten GHz-Tagung (ich meine von Uwe, DL1SUZ) geschildert wurde, dass das automatisierte Anschreiben mehr nervt, als nützt, würde ich so unterschreiben. Dort fehlt eine Heuristik, schon angeschriebene Stationen mit „negative signals“ beim nächsten Nachrichtenversand auszulassen. VUSC kann das meines Wissens nach nicht. Mass-Mailings beherrscht es aber.

Auch das ist ein Umstand, den ich mit KST4Contest explizit vermeiden wollte. Es gibt jetzt explizite Wege zur Vermeidung von Spam.

Chat-Mining: Jagdinstinkte wecken mit KST4Contest

5) KST4Contest v1.31 – Update

Nachdem der theoretische Unterbau glücklicherweise brauchbare Ergebnisse lieferte, konnte damit begonnen werden, konkrete Änderungen zu planen und zu implementieren. Die Implementierung überholte zuweilen den Unterbau. Der ist zäh und zeitaufwendig.

Ich gehe wieder vor, wie beim letzten Mal, zeige einen Screenshot und erläutere dann, was sich geändert hat, warum – und wie.

The screenshot displays the KST4Contest v1.31 software interface. The window title is "Connected to 2: 144/432 MHz and 3: Microwave as DOSAMF (KST4Contest131) in jn49FL (102 users online, 102 shown), 1685 messages total." The interface is divided into several sections. At the top, there's a menu bar with options like File, Options, Windows, Info, UDP-Log msg on, Wintest-msg on, MessageBus on, PSTRotator on, DXCluster-Server on, MyBeacon off, AirScout msg on, and SkedReminderService on. Below the menu bar, there's a status bar showing "Show only QRB (km) <= 1333.0", "Show only QTF: 135.0", "deg +/- 66.0 N", "deg +/- 66.0 E", "deg +/- 66.0 S", "deg +/- 66.0 W", "deg +/- 66.0 NW", "deg +/- 66.0 NE", "deg +/- 66.0 SE", "deg +/- 66.0 SW", "deg +/- 66.0 NW". The main area is divided into two panes. The left pane shows a chat log with messages from various users. The right pane shows a table of active stations. The table has columns for Callign, Name, QRA, QRB, QTF, QRG, Act, AP (minutes), worked, NOT QRV, and Category. The table lists several stations, including M7JFY, OE3F, OE6MB, and OE7UEB. The bottom right corner shows a detailed view of a station's activity, including QTF, QRB, and QRG values, and a "Show path in AS" button.

Der UI-Aufbau der Software und die Optimierung hinsichtlich Workflow geht aus den Unterlagen der GHz-Tagung 2025 hervor. Wer ihn noch nicht kennt, sollte ggf. das Skript lesen.

Für diejenigen, die die UI und/oder das Programm schon kennen, sind die Änderungen augenfällig. Es könnte sein, dass sich die Anordnung der einzelnen UI Komponenten noch einmal ändert.

Die Marschrichtung für die Weiterentwicklung des Chatclients war klar:

Sked-Anfragen mussten verbessert und stärker an den statistisch erfolgreichen Sked-Requests ausgerichtet werden. Ziel war es nicht, seine Nachrichten trivial nach einem Merktzettel zu tippen.

Das wichtigste Ziel war, Informationen aus dem Chat in Echtzeit aus den Texten abzuleiten, für die ein Chat-Operator normalerweise Unmengen an Zeit bräuchte.

Diese Informationen müssen dem Operator dienen, ohne, dass er einen Zusatzaufwand bemerkt.

Was tun? Sprach Zeus.

Chat-Mining: Jagdinstinkte wecken mit KST4Contest

a) Entwicklung eines KST-Simulators

Mit **DL8AAU** (wtKST) und **DL2ALF** (AirScout) führte ich bei einer abendlichen Halben in Dorsten ein interessantes Gespräch über das Machbare in Sachen Software. Vorstellungen entwickle ich dabei immer schnell, die Umsetzung dauert oft eine Weile länger, als es mir lieb ist.

Eine Annahme von DL8AAU war, dass ich einen KST-Server-Simulator hätte. Man müsse die Funktionen ja in einem echten Umfeld testen und könne ja nun wirklich nicht alles live auf ON4KSTs Server testen.

Nun ja. Bis Mitte 2025 machte ich das noch immer genau so (wobei Tests einzelner Methoden und Funktionen anderswo automatisiert waren/sind). Ich erwähnte das an dem Abend nicht und werde es nach dem Schreiben dieser Zeilen auch direkt abstreiten...

Für die folgenden Entwicklungen war es unumgänglich, einen richtigen Test-Server zu implementieren, um den Chatclient einer Simulationsumgebung mit plausiblen, simulierten Daten auszusetzen. In 31 Iterationen entwickelte ich einen Testserver, der wenig glaubwürdigen, aber nahe an der Realität gebauten Datenmüll produziert, mit dem ich testen kann. Darunter fallen auch Frequenzangaben deren Stabilität, Antwortintervalle, Stresstests, Fehleingaben der Nutzer etc.

b) Win-Test support

Auf mehrfachen Wunsch einiger Nutzer kümmerte ich mich um das Thema „Win-Test“-Support. Anfang letzten Jahres schrieb ich die Entwickler an und fragte nach, ob sie das UDP contactinfo-Interface einbinden können, wie es ca. 90% der anderen Logger tun. Ich bekam eine Zusage. Einige Monate später fragte ich nach dem Stand. Ich bekam die Info, dass das Feature nicht gerade hoch priorisiert ist. Damit konnte ich leben.

Ich wartete weitere Monate: vergebens. Ich verwies KST4C-Nutzer im Verlaufe eines Jahres darauf, das Feature bei den Entwicklern anzufragen. Die Antworten blieben jeweils aus oder waren negativ. Auf zwei eigene Nachfragen bekam ich überhaupt keine Antwort. Solche Erfahrungen mit dem Win-Test-Support sind offenbar nicht neu, wie ich später erfuhr.

Da der Wunsch nach Win-Test-support bei den Usern weiterhin bestand, ging ich vor, wie früher bei UCXLog (**dessen Entwickler mit großem Abstand offener für Nachfragen ist**). Ich installierte Win-Test und analysierte dessen Netzwerk-Protokoll. Ergebnis: die Win-Test-Interne Kommunikation gleicht technologisch stark der des ContactInfo-Interface. Ich konnte diese Pakete daher einfach verwerten. Die Implementierung durch Win-Test selbst wäre vermutlich noch schneller gegangen.

Ich selbst mag die Software nicht besonders und verstehe auch nicht, warum sie so viele Oms benutzen. Daher habe ich das Thema deren Integration lange vor mir her geschoben. Nach der Erfahrung mit dem Support werde ich wohl auch weiterhin kein Win-Test-Fan mehr werden.

Meine letzte Mail an das Team, dass und warum das Thema erledigt ist, blieb auch ohne Antwort.

Chat-Mining: Jagdinstinkte wecken mit KST4Contest

c) Zweite Chatkategorie

Diese Änderung ist eine direkte Folge aus den Analysen zur Anwesenheit der Chat-Teilnehmer.

Beim Betrachten des ersten Bildes ist sicherlich aufgefallen, dass es in allen Nachrichtentabellen ein neues „Category“ Feld gibt, hier im Bild nochmals zu sehen.

Auch in der Userliste kam das neu dazu. Schon im März 2025 integrierte ich aus o.g. Gründen die Möglichkeit, eine zweite Chatkategorie zu nutzen.

Callsign	Name	QRA	QRB	QTF	QRG	Act	AP (minutes...)	worked	144	432	23	wkdany	NOT QRV @	Category
LY5ZS	Alain	KO86MY	2058	55.1°	10368.181	0	nil							Microwave
LY7WS	Steve	KO05EB	1018	47.95°	432.599	0	nil							Microwave
LY8DW	Mario	KO35ZY	1502	53.59°	432.258	0	nil							144/432 MHz
LZ1D	Giovanni	KN29OB	1213	85.8°		1	nil							Microwave
LZ3KNE	Hans	KN23TS	1452	109.39°	10368.189	0	nil							Microwave
M31T	Jo	JO16FU	911	336.37°	1296.294	2	22 (50%)							Microwave
M7X	Sven	JO16AJ	880	333.32°	144.272	0	3 (75%) / 1...							144/432 MHz
OE3F	Mike	JN74HK	729	137.85°	144.599	0	13 (75%) / ...							144/432 MHz
OE6MB	Bob	JN00JP	1150	214.3°	10368.204	0	nil							Microwave
OE7UEB	Mario	JN80EE	1205	145.99°	1296.111	0	nil							Microwave
OE7UEB	Mario	JN80EE	1205	145.99°	432.17	0	nil							144/432 MHz
OE8Y	Hans	JN45HC	486	178.46°	10368.138	0	0 (100%) / ...							Microwave
OE8YK	Peter	JN34VW	507	185.94°	144.599	0	0 (100%) / ...							144/432 MHz
OK0J	Peter	JO35ZO	681	358.24°	144.599	0	nil							144/432 MHz

Ich wollte den Weg, den ich vorher eingeschlagen hatte, nicht verlassen. Der Chat-OP sollte von den Kategorien abstrahieren können. Aus meiner Sicht ist es ausdrücklich nicht wünschenswert, zwei Chatfenster zu beobachten oder zwei Nachrichtenverläufe nachzuvollziehen. In meinem Kopf gehören diese Daten zusammen.

Aus diesem Grund kann es jetzt Duplikate in der Userliste geben, falls eine andere Station sich mit dem gleichen Rufzeichen zweimal anmeldet. Der Unterschied liegt dann **nur** in der Kategorie. Wenn ich eine Nachricht gesendet bekomme, sie anklicke und darauf antworte, sendet der Client sie automatisch an **genau den richtigen Absender** zurück. Es gibt also ein Autorouting für die Nachrichten. Ist nicht klar, wo die Nachricht genau hin soll, wird sie in der Kategorie gesendet, in der der Adressat zuletzt sendete.

d) QSO-Sniffer

Eingangs erwähnt und auch schon in der letzten GHz-Tagung auf der Roadmap war das Thema, Chats von anderen Stationen abzufangen. Unter anderem **9A2HM** bat mich, das Thema nochmal aufzugreifen und nachdem er nicht der erste war, der das gern haben wollte, implementierte ich es. Da ich selbst weder den Wunsch noch die Vorstellung nach so einem Feature hatte, implementierte

ich es so, wie mir unter anderem von SM6VTZ vorgeschlagen wurde und wie im Bild zu sehen. Die fremden Nachrichten landen in meinem Posteingang. Die Implementierung dauerte abenteuerliche 15 Minuten.

Chat-Mining: Jagdinstinkte wecken mit KST4Contest

e) Smart Frequency-recognition

Das erste Ziel und der Hauptgrund, für die Entwicklung von KST4Contest war, auf Basis der Chat-Daten eine Bandmap aufzubauen. Die Erkennung von Frequenzstrings über regular Expressions kostete mich eine ordentliche Portion Nerven und war ab Version 1 unverändert implementiert.

Etwas, was mich schon von Anfang aufregte, war die Unsicherheit darüber, zu sagen, aus welchem Band eine Frequenzangabe stammte. Operator-Angaben wie „**pse try .210**“ treiben einen als Entwickler in den Wahnsinn. Die .210 könnte in einem Multiband-Contest in jedem Band liegen.

Das war aber nicht alles. Es wurde auch nur die letztgenannte Frequenz zu einem Teilnehmer gespeichert. Die einzige Art Historie, die es für QRG Strings gab, war der eingebaute DX-Cluster Server, der mittels „sked-directed-to-me-highlighting“ die Frequenzen zumindest eine Zeit lang konservierte.

Eine neue Idee zur Workflow - Optimierung entstand.

Wäre es nicht interessant, zur Laufzeit zu wissen, ob eine selektierte oder gerade gearbeitete Station **außer auf dem aktuell genutzten Band noch auf einem weiteren Band** qrv ist, was ich auch unterstütze?

Angenommen, ich handle mit so einer Station einen Sked aus, dann sehe ich, dass noch weitere Bänder gefunkt werden können und hänge diese direkt an den ersten Kontakt an. Das verspricht enormes Punktepotezial und das Beste für die Misanthropen unter den Funkamateuren ist: man muss gar nicht nachfragen, ob man ein anderes Band funken kann, sondern schlägt gleich die letzte QRG des Gegenübers vor. Es war Zeit, dem alten Ding ein Update zu verpassen.

Es folgt noch ein Prozessbild. Vereinfacht gesagt ist es so: ich speichere jetzt nicht mehr einen Frequency-String pro Station in das Stations-Objekt, sondern das Stations-Objekt enthält eine Map<QRG>, sodass ich beliebig viele festgestellte Frequenzen in das Stations-Objekt speichern kann. Aus den Frequenzen gehen die Bänder hervor, die unterstützt werden, sodass ich mir einfach eine Warn-Mechanik bauen kann. Diese kann z.B. beim Loggen einer Station direkt den Chat-Op warnen, dass da noch ein weiteres Band möglich ist.

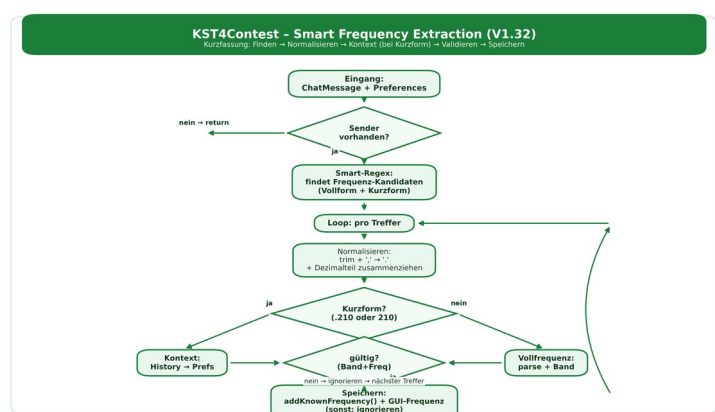


Abbildung 5.1: smart-frequency-extraction

Was diese frequency-extraction smart macht ist, dass beim Empfang von Angaben wie „.210“ innerhalb der Map geschaut wird, ob in den letzten 30 Minuten eine „vollständigere“ Frequenzangabe gefunden wird (z.B. 144.210), die das gleiche Suffix (aber auch

Chat-Mining: Jagdinstinkte wecken mit KST4Contest

ein Präfix) hat. Ist dies der Fall, dann wird hier z.B. die 144 ergänzt. Die Frequenzangabe ist damit viel zuverlässiger als vorher und in der Regel auch vollständig, ohne dass das Präfix geraten werden muss. Ein dadurch entstehender Nebeneffekt ist, dass mehr KST4Contest-interne Clustermeldungen erzeugt werden können, weil das nämlich nur dann passiert (und Sinn ergibt), wenn die QRG vollständig erkannt wurde.

Von Rufzeichen-Prefixen und -Suffixen (z.B. -2 oder -70) wird außerdem abstrahiert. Die Frequenzangaben werden nur für das normalisierte Rufzeichen gespeichert und in allen Rufzeichen-Objekten mit diesem Raw-String hinterlegt.

In der Simulation klappt das gut, in der Praxis muss es sich erst noch beweisen. Wenn das mal keine gezielte Sked-Anfrage möglich macht?

f) Priority-Score und seine Folgen

Jetzt wird es endlich mal wieder wirklich innovativ, finde ich.

Der grundlegende Aufbau der allermeisten existierenden Chatclients mit Chatroom-Betrieb ist gleich und stützt sich auf die Userliste innerhalb eines Channels. Der gut abgestandene Funkamateurliebt das von Chatprogrammen wie MIRC (aus den guten alten IRC-Zeiten) und AOL Chatrooms.

Es spricht nichts gegen dieses Prinzip; sehr praktisch ist es aber nicht. In Zeiten von IRC und den AOL Chatrooms, trainierten wir uns in unseren Jugendtagen das schnelle Tippen an und es machte beinahe keinen Unterschied, wem man im Chatroom „eine Klingel ans Knie laberte“.

Chat-Mining: Jagdinstinkte wecken mit KST4Contest

Der Nutzer sitzt jedenfalls vor einer Liste und muss die Entscheidung treffen, wen er als nächstes anschreibt. Das ist, übertragen auf den KST Chat, auch so. Dazu nutzt er eine Heuristik in seinem Kopf, welche Station für einen Sked als nächstes am sinnvollsten sein könnte. Hin und wieder hilft der Broadcast-Chat. Auch günstige Flugzeug-Querungen können so eine Entscheidung unterstützen, müssen aber in der Userliste nachgeschlagen werden.

Ich habe bei mir selbst die Beobachtung gemacht, dass das mit Länge des Contests nicht einfacher wird und daher entschieden, dass die in der Userliste befindlichen Stationen einen **Bewertungsmaßstab** brauchen, nach dem objektiv bewertet werden kann, welcher Sked-Partner als nächstes angeschrieben werden sollte. Ich nenne das „Priority Score“ anhand „Stationsmetrik“. Die Userliste wird zur Datenbasis für das Programm, nicht mehr zum Dreh- und Angelpunkt für manuelle Nutzerentscheidungen. Es muss sich erst noch bewähren aber meine aktuelle Vorstellung ist, dass man die Userliste nur noch dann durchsucht, wenn einem sehr, sehr langweilig ist.

Die Score-Berechnung ist ein größerer Prozess mit einigen Eingabeparametern, der von einigen Programmteilen getriggert wird. Für eine saubere Implementierung der Logik werden Stationsmetriken erstellt, die in einem Score Snapshot gesammelt werden und dann, wenn ein Recompute des Scores aufgrund bestimmter Änderungen getriggert worden ist, in der UI aktualisiert. Ein Prozessdiagramm ist oben zu sehen, die Faktoren, die zur Score-Berechnung dienen, unten. Das Fenster zeigt die resultierende Priority List, die meinem kühnen Wunsch nach die **Userliste ablösen** wird. Für M7JFY habe ich vorher einen manuellen Sked erstellt, weshalb er sehr weit oben in der Liste steht. Zeitkritische Situationen erkennt der ScoreService und bringt solche Stationen in der Liste ganz nach oben. Außerdem werden Aktivität, Antwortverhalten, Ruf-Richtung, „positive Signals“ (Textanalyse), worked-Status pro Band, AS-Status, Beamrichtung und Distanz bewertet. Eine **mittlere Distanz** wird dabei höher

Score / Priority: Prozess (KST4Contest)

kurz, ohne Details — Stichworte für deine Erklärung

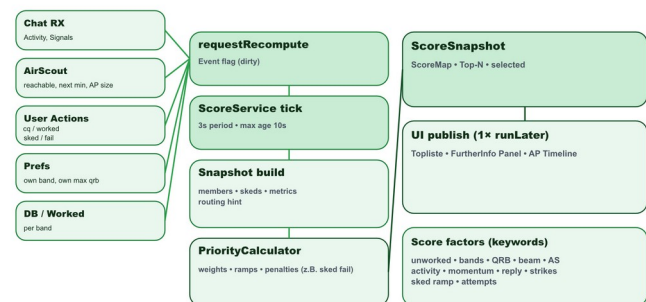


Abbildung 5.2: Score-Prozess

Score-Bewertungsfaktoren (KST4Contest)

Stichworte - Kurzbeschreibung - basiert auf der aktuellen Priority/Score-Logik

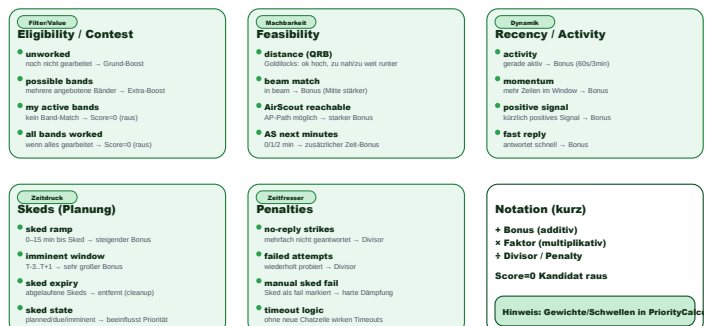


Abbildung 5.3: Score-Faktoren

Top priority candidates	
M7JFY	score 1570
2E6UG	score 1119
SP8C	score 1052
DL1TEST	score 1037
9AS9B	score 1037
DO8XQ	score 1037
GM8RY	score 1037

Chat-Mining: Jagdinstinkte wecken mit KST4Contest

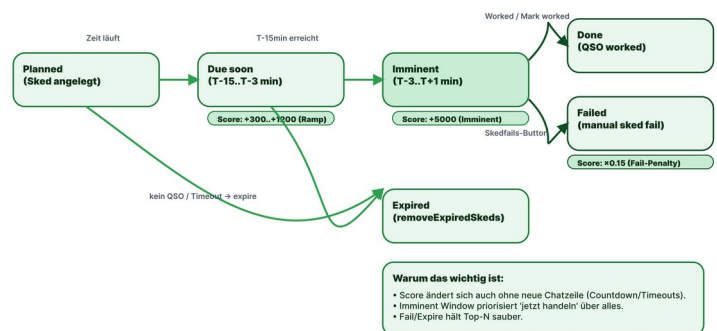
bewertet, als eine Hohe oder Niedrige, weil ein maximales Punkte/Aufwandsverhältnis bringt, da sie in der Regel schnell realisiert werden kann. Sked-Fails reduzieren den Score, ausbleibende Antworten auch. So wird aktiv vermieden, dass Stationen priorisiert angeschrieben werden, von denen eigentlich bekannt sein könnte, dass der Pfad nicht funktioniert oder sie sowieso nicht antworten (wollen).

g) Skeds

Ein Feature, was ich selbst immer vermisst habe, ist, dass ich mich an Skeds erinnern lassen kann. SP2SOO ist von JO51IJ aus zum Beispiel nur ca. alle 30 Minuten per Flugzeug zu erreichen. Es gibt kaum günstige Gelegenheiten. In so einem Fall, dass ein Flugzeug noch lange hin ist, verspricht man dem Gegenüber (und sich selbst) gern, zu erinnern, wenn das ersehnte Flugzeug endlich da ist. Im Eifer des Chat-Gefechtes verschläft man es dann aber doch. Das ist erstens unfein und zweitens nervt es einen ja auch selbst, wenn die Gelegenheit auf ein Flugzeug einfach verstreicht. Was wäre da naheliegender, als ein **Sked-Timer**? Und zwar einer, der auf Wunsch nicht nur mich erinnert, sondern auch die zu funkende Station mit rechtzeitigen Nachrichten?

Sked-Lifecycle: Zeitfenster → Aktionen → Score-Effekt

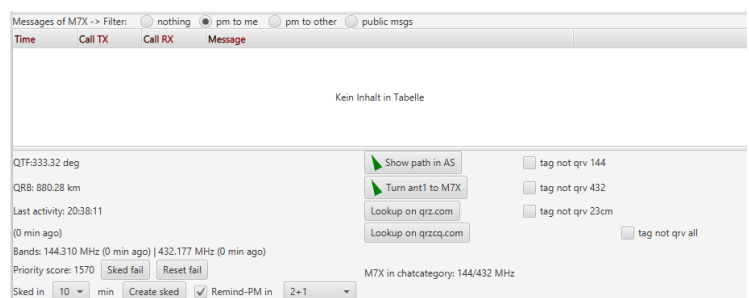
Skeds steuern die Zukunft im Chat (Ramp + Imminent Boost, plus Fail/Expire).



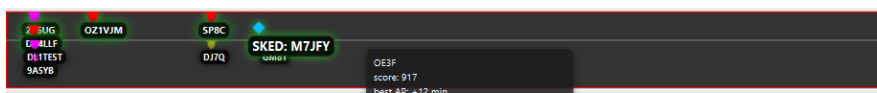
Editorbar: SVG in Inkscape / LibreOffice Draw / PowerPoint öffnen.

Abbildung 5.4: Sked-Prozess

So eine Funktion finden wir im Fenster rechts. Die Sked-Zeit ist in Grenzen selektierbar. Nach einem Klick auf „create sked“ wird der **Kurzzeitwecker** scharf geschaltet. „Remind-PM in“ wird aktiviert, wenn gewünscht wird, dass der andere Operator in n Minuten (in diesem Fall 2 Minuten + nochmal 1 Minute vor dem Sked-Termin) per PM auf das Eintreten des Sked-Fensters hingewiesen wird.



Nach „Create“ Sked bekommt die Station in bestimmten Zeitintervallen sehr hohe priority-boosts, sodass sie die Score-Liste anführt, je nach dem, wie lange der Sked noch in der Zukunft liegt. Es gibt akustische und optische Warnungen für das Eintreten eines manuell veranlassten Skeds. So verpasse ich hoffentlich keinen Sked mehr und SP2SOO hoffentlich auch nicht.



So sieht ein Sked in der Timeline aus (noch ca. 7 Minuten)

Chat-Mining: Jagdinstinkte wecken mit KST4Contest

h) Aircraft-Scatter timeline

Ein inniger Wunsch, den **DL8AAU** nach der letzten GHz Tagung beim Genuss der oben erwähnten Halben äußerte, war sinngemäß, dass es schön wäre, wenn man sich **Partnerstationen anhand der Flugzeuge** „herauspicken“ könnte. Ich sofort Ideen zu dem Thema und einige befinden sich in Umsetzung. Das Thema kann recht komplex werden, je nach dem, wie man es angehen möchte.

Meine Ursprungsidee sieht vor, eine Matrix aus Stationen zu erstellen und als Eigenschaften der Stationen Sets von Gegenstationen zu erstellen, die Aircraft Scatter Daten enthalten. Aufgrund Komplexität und Rechtslage wird die Berechnung der Flugbahndaten dezentral als Cluster-Computing von den verteilten KST4Contest Clients realisiert. Die Daten werden bei hamradioonline.de in einem Service verwertet, gecached und für andere Chatclients verfügbar gemacht. Funktioniert (ist aber nicht aktiv!).

Ein Problem dabei sind die oft antiquierten Rechner der Funkamateure. Die AS-Rechenaufgaben werden vom cluster computing master an die Clients herausgegeben und deren Ergebnisse verwertet. Wie viele Bahndaten die Clients ausrechnen können, ist sehr, sehr unterschiedlich und in der Ursprungsimplementierung zu stark vernachlässigt worden. Ich selbst habe naturgemäß eine relativ leistungsfähige Maschine für die Entwicklungsarbeit zu Hause. Andere haben das nicht. Die Rechenaufgaben dauern zu lange und es kommt zu Synchronisationsproblemen zwischen AirScout und Chatclient, weil AirScout die Ergebnisse mit der Zeit viel später zurückliefert, als erwartet. Es scheint eine Computingrequest-Queue zu geben.

Eine Lösung dafür habe ich konzipiert aber bisher nicht umgesetzt. Sie hat mit dem Timing der Rechenaufgabenverteilung zu tun. Das ist nicht so einfach, weil AirScout für solche Kopfstände nicht gedacht ist. Der Gesamtrechenaufwand ist nicht das Problem, eher die Verzögerung der Einzelaufgaben.

Lange Rede, kurzer Sinn: **es geht auch einfacher.**

Die schnelle, lokale Lösung ohne große Nutzergemeinschaft, nenne ich **Aircraft Scatter timeline**. Die Timeline ist ein Zeitstrahl, in dem die High-priority-score Stationen in Antennenrichtung dargestellt werden. Grundlage der Darstellung ist die Integration von AirScout-Daten.

Zu jeder abbildbaren Minute werden die high priority Stationen dargestellt, wenn ein Aircraft-Scatter-Fenster zu ihnen besteht. Da es sehr, sehr oft viele Flugzeuge zu einer Zielstation gibt, werden in der Timeline bevorzugt die Flugzeuge dargestellt, die ein hohes Reflexionspotenzial aufweisen. Nur, wenn es keine hohen Potenziale gibt, werden andere Flugzeuge mit ihren höchst priorisierten Stationen dargestellt. Prinzipiell immer maximal vier pro Minute gleichzeitig.

Airplane-Scatter Timeline: Prozess

Preview 30min • per Minute Buckets • 2x in-beam • 2x global distinct

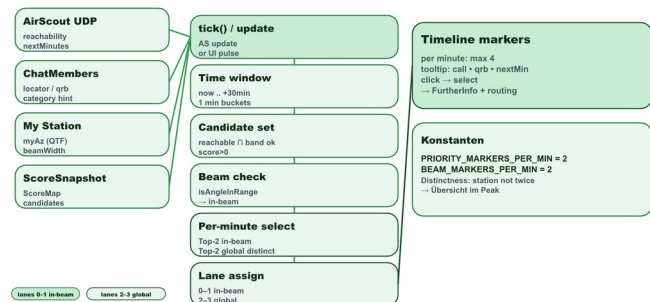
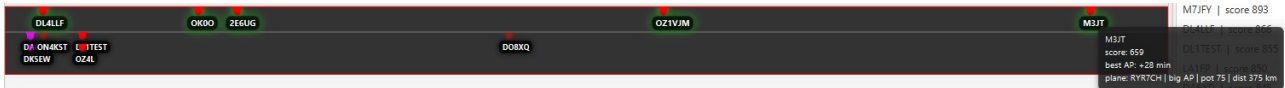


Abbildung 5.5: AS-Timeline: Prozess

Chat-Mining: Jagdinstinkte wecken mit KST4Contest

Die Timeline stützt sich zusätzlich auf das PSTRotator Interface. Steht der Rotor innerhalb der definierten Beamwidth-Parameter auf eine Station in der Timeline, dann leuchtet diese recht stark auf. Ein Beispiel dafür ist in folgendem, Bild zu sehen. DO8XQ liegt als Einziger nicht im Beam.



Steht der Rotor nicht dort hin, erscheint die Station transparent (DO8XQ).

Die Farben der Marker („crystals“) repräsentieren das Reflexionspotenzial des Flugzeuges. Die Lanes (es gibt 4) repräsentieren die Berücksichtigung der eigenen Rotorposition (siehe Diagramm).

Dadurch, dass der Priority Score selbst auch schon vorsieht, ob die Station zuletzt in meine Richtung für einen Sked angefragt hat und die Frequenzen dieser Station meist bekannt sind (da auch Teil des Priority Scores), sollte sich die Möglichkeit ergeben, diese Stationen der Reihe nach durch zu klicken und „weg zu funken“. Der März wird zeigen, wie gut das funktioniert.

Auch hier sehen wir direkte Folgen der eingangs angestrebten Betrachtungen. Es ist möglich, (gemäß der Heuristik) **hochwertige Sked-Anfragen** zu senden, die bereits alle Daten enthalten, um sofort loslegen zu können.

i) PSTRotator-Interface

Letztes Jahr erwähnte ich schon, dass ich das Interface integriert habe aber keinen solchen Rotor besitze. Ungeplant habe ich das Thema wieder aufgegriffen, nachdem mein guter Freund Jörg, **DK8DE** und ich uns sehr ausgiebig mit Auf- und Umrüstung von Rotoren befasst haben (neben anderen Projekten wie dem „pro buffer“ getauften Sequencer mit CW-Pufferfunktion). Beides bietet Material für einen separaten Vortrag und wird deshalb hier nicht weiter präzisiert. Der Sequencer wird von Jörg in Dorsten 2026 vorgestellt.

Ab dem nächsten Release ist das Rotor-Interface aktiv. Es ist getestet mit meinem alten Spid RAS. Es sind auch Unwägbarkeiten aufgefallen und bereinigt worden, die Spid in seinem Protokoll gratis mit liefert. Als Ergebnis haben wir die Auswirkung auf ein harmloses Feld rechts neben der Texteingabezeile und Nebenwirkungen wie die Auswirkung auf die AS-Timeline und die user-priority-table. Die sichtbaren 360° sind der Initialisierungswert des Rotors. Links daneben die CAT2-QRG, links daneben die CAT1-QRG.



Es wäre auch denkbar, die Filter für die Userliste automatisch nach Rotor-Stellung zu aktivieren, das ist aber aktuell nicht implementiert und gehört ggf. auf die Roadmap.

j) Auto-Answer

Letztes Jahr erwähnte ich, dass ich beim Loggen einer Station eine **auto-73** Nachricht an die Gegenstationen senden könnte, die eine Prüfsumme für den Cross-Check enthält. Ein Raunen

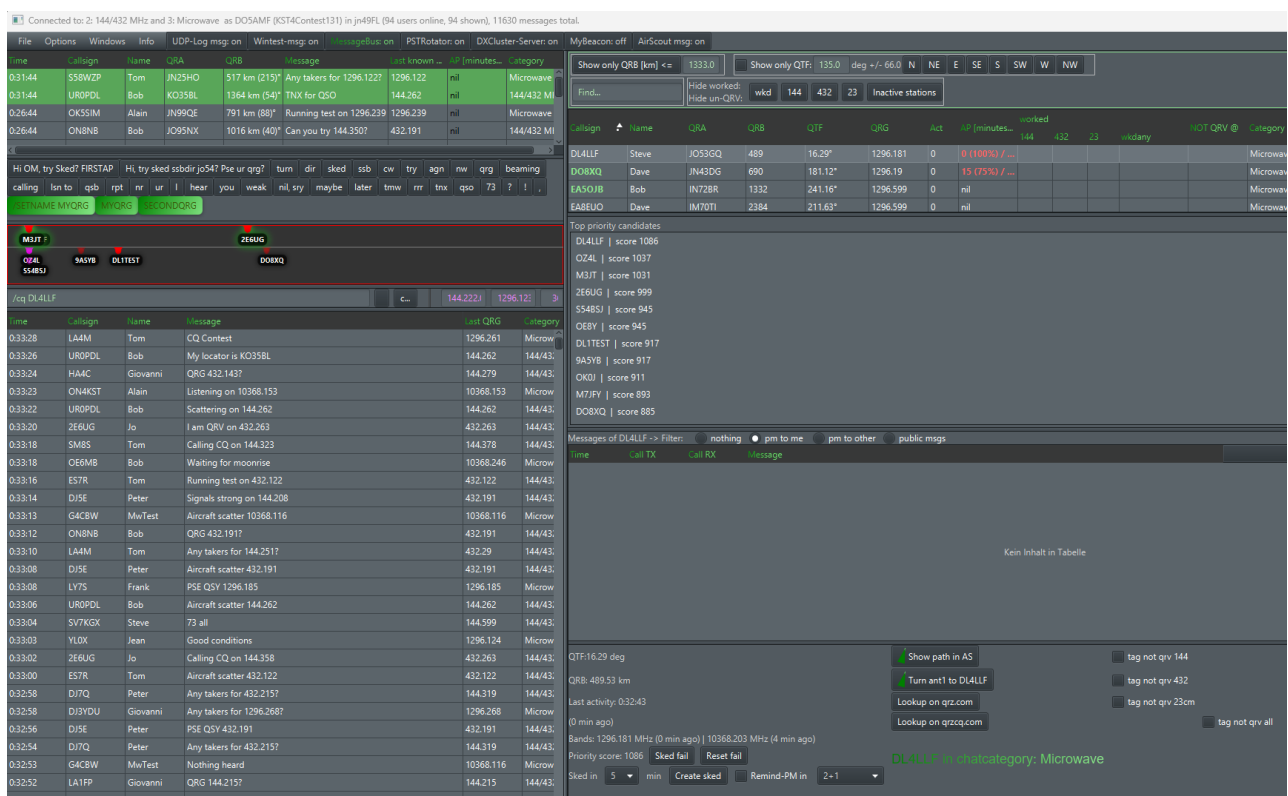
Chat-Mining: Jagdinstinkte wecken mit KST4Contest

erfüllte den Raum. Es ist schon zu lange her aber ich meine, der Tenor war, dass das nicht den Contestregeln entspräche. Ich habe daher auf die Implementierung eines solchen Features verzichtet, wie auch auf Mass-Mailingfunktionen. Ein 73-tnx Paket als Bestätigung wäre nach den eingangs erwähnten Analysen, trotzdem eine nette Idee.

Auch das habe ich nicht implementiert. Was ich implementiert habe, ist eine **Auto-Answer für QRG-Nachfragen** und eine für **alle eingehenden Nachrichten**, für Leute, die nicht gestört werden wollen. QRG-Nachfragen können damit automatisiert beantwortet werden, KST4Contest kann auf übliche Phrasen mit beiden QRG antworten.

k) Dark Mode

Das stand letztes Jahr schon auf der Roadmap. Der Dark Mode gehört heute beinahe überall zum guten Ton. Hier ein kleines Bild davon, wie das aussieht:



Chat-Mining: Jagdinstinkte wecken mit KST4Contest

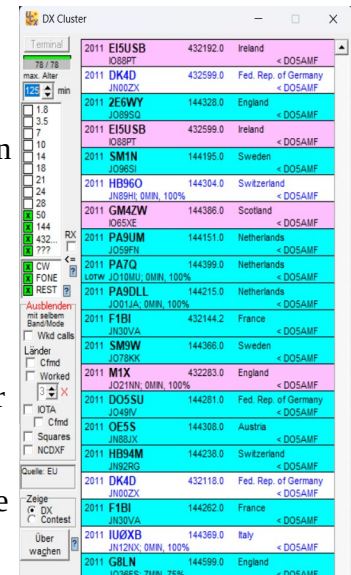
I) DXCluster Server, AP Info Integration

Der DXCluster Server informiert das Contestprogramm wie ein normales Cluster. Das Feature hatte ich letztes Jahr schon drin. Die Rückmeldungen sind sehr positiv und der angestrebte Ablauf „Anklicken, funken, weiterklicken“ ergibt sich tatsächlich für immerhin ca. 50% aller Spots (je nach Bedingungen).

Schließlich werden die Spots durch die sked-directed-to-me Mechanik getriggert.

Im letzten Release habe ich neben dem Locator im Spot auch noch die Information des nächsten AP mit eingefügt. So hat der Spot-Empfänger das maximale Maß an Information.

Im Bild: DXCluster Meldungen in UCXLog. Ein klick triggert auch die QRG-Änderung des TRX, sodass man bequem über potenzielle Gegenstationen iterieren kann.



Call	Frequency	Country	AP
2011 EI5USB	432192.0	Ireland	< DOSAMF
2011 DK4D	432599.0	Fed. Rep. of Germany	< DOSAMF
2011 Z6WV	144328.0	England	< DOSAMF
2011 EI5USB	432599.0	Ireland	< DOSAMF
2011 SM1N	144195.0	Sweden	< DOSAMF
2011 HB960	144304.0	Switzerland	< DOSAMF
2011 GM4ZW	144386.0	Scotland	< DOSAMF
2011 PA9UM	144151.0	Netherlands	< DOSAMF
2011 PA7Q	144399.0	Netherlands	< DOSAMF
2011 PA9DLL	144215.0	Netherlands	< DOSAMF
2011 F1BI	432144.2	France	< DOSAMF
2011 SM9W	144386.0	Sweden	< DOSAMF
2011 M1X	432283.0	England	< DOSAMF
2011 DO5SU	144281.0	Fed. Rep. of Germany	< DOSAMF
2011 OE5S	144308.0	Austria	< DOSAMF
2011 HB94M	144238.0	Switzerland	< DOSAMF
2011 DK4D	432118.0	Fed. Rep. of Germany	< DOSAMF
2011 F1BI	144282.0	France	< DOSAMF
2011 IUGXB	144369.0	Italy	< DOSAMF
2011 G8LN	144599.0	England	< DOSAMF

6) Integration der Software in den Contestablauf

Ich zitiere mich selbst vom letzten Jahr:

„Wie zwischenzeitlich formuliert, unterscheidet sich die Art und Weise zu contesten heute stark von der, die ich zu Anfang meiner Contestlaufbahn erlebte. Ich bin Informatiker und habe daher einen intrinsischen technologischen Spieltrieb. Ich kann aber gut die Gegenstimmen der Contester verstehen, die ohne Chat auch erfolgreich waren/sind und ihren Funkbetrieb nicht von einem anderen Medium, als vom Funk, abhängig machen möchten.

Der Chat ist eine Sache, auf die man sich einlassen muss. Das ist ein Prozess, der unserem Hobby inne wohnt. Für die Contester unter uns ist eine 24 Stunden lange Veranstaltung der ideale Zeitpunkt, um die eigene Belastbarkeit und die der Komponenten unter Beweis zu stellen.“

Das habe ich postuliert und das soll auch in diesem Jahr mein Résumé sein. Auch, wenn meine statistischen Herleitungen nicht so lupenrein dargelegt sind, wie sie mein gern gemochter Prof. Dr. Frank Nürnberg bestimmt gern sehen würde, so halte ich es doch für nachweisbar, dass der Einsatz des Chats einen nennenswerten Einfluss auf die Contestergebnisse hat. Selbst, wenn eine gute Platzierung eher ein Nebeneffekt ist, so ist es gewiss kein Fehler, möglichst viel Effizienz bei der Verarbeitung der Skeds walten zu lassen.

PS: Die Quell-Dateien zu den Betrachtungen aus den ersten Seiten stelle ich selbstverständlich gern zur Verfügung, wenn sich jemand dafür interessiert.

Chat-Mining: Jagdinstinkte wecken mit KST4Contest

Ich freue mich auf den März-Contest, um die neuesten Features im Realbetrieb testen zu können und bin gespannt, ob der Einsatz des Clients eine Auswirkung auf unseren Score haben wird.

7) To-do-Liste

An der Stelle gibt es eine Kopie aus meinen aktuellen Arbeitsunterlagen.

- Zeitfenster-Analyse (war letztes Jahr auch schon drin)
 - Warnung im Client, dass eine Station qrv ist, die üblicher Weise nur kurz verfügbar ist
 - Das Konzept ist fertig, geht aber nicht ohne Server, das wird noch etwas dauern
- Logbook-History-analyse
 - Warnung im Client, dass eine Station qrv ist die man üblicher Weise schon gefunkt hat
 - DK5OH schlug das vor und es ist eines der nächsten Ziele
- Line wrap (auch vom letzten Jahr)
 - Zeilenumbruch für lange Texte + Kopierbarkeit des Textes
- „Auto-73“, nur zur Bestätigung
 - Ist das eher akzeptabel?
- „Back to CQ-QRG“-spot after QSY
 - Nach QSY erscheint ein interner self-Spot mit der alten CQ-QRG, damit der TRX wieder auf die CQ QRG wechseln kann, ohne ihn anzufassen
 - Es sind weitere Spots geplant, z.B. Spot-on-Sked, um QRG zu automatisieren
- Protocol and webservice for chatusers QRG and qrv-band exchange
 - Arbeit mit G1YFG zur Entwicklung eines Austauschprotokolls für QRG Daten
 - Das ist im pending aus oben erwähnten Gründen.
- Airplane-Filter und Category-Filter für die Userliste
- Locator-Sniffer und Keyword-Sniffer
- Lifetime für Logged-Flags
 - Aktuell muss der Nutzer immer noch selbst resetten, das muss raus
- Manueller worked-Button
 - Falls ein UDP-Paket mal verloren ging...

Chat-Mining: Jagdinstinkte wecken mit KST4Contest

8) ZDF - Zahlen, Daten, Fakten

a) Entwicklungszeit und Aufwand

Der Chatclient wird jetzt seit Mitte 2022 entwickelt. Regelmäßig dann, wenn die Kinder im Bett sind, maximal 2-3 Stunden am Tag. Es sind ohne jede Übertreibung **hunderte Stunden** in die Entwicklung geflossen. Die Erstellung des User Manuals klingt trivial, kam in der ersten Variante des Chatclients auch schon auf **22 Stunden** (netto!). Das Manual braucht ein umfassendes Update, da hier seit ca. einem Jahr nichts mehr passiert ist.

b) Danke

Dass der Chat die Entwicklungsumgebung verlassen hat, verdanke ich **Konrad Neitzel, DC9DJ**, von dem ich erst erfahren habe, dass er Funkamateurliebling ist, nachdem ich mich mit dem Deploy-Problem an ihn als Berufs-Softwareentwickler des Java-Forums wandte. Dafür bin ich sehr dankbar!

Es gibt außerdem Nutzer, die die Software regelmäßig vor der Veröffentlichung testen und die Ergebnisse zurückmelden. Vielen Dank dafür!

c) Umfang

Aktuell besteht das Gesamtprojekt aus ca. **45.000 Zeilen** Code (es ist tatsächlich seit letztem Jahr kaum mehr geworden, da ich auch viel refaktoriert habe). Die Nutzergemeinde steigt stetig an und befindet sich (nur über Updatelink-Abrufe nachvollziehbar) schon einige Zeit im dreistelligen Bereich. Ich bekomme auch regelmäßig Anfragen für neue Ideen und Feedback für Änderungen, für die ich auch sehr dankbar bin.

d) Reich wird man damit nicht

Da ich den Client in erster Regel für DM5M und dort hauptsächlich für mich entwickle, spielt es keine große Rolle. Aber weil es heißt: „über Geld spricht man nicht“ und ich ein Rebell bin, mache ich es natürlich trotzdem. **Stand 01/2026** beträgt die Anzahl der Spendenbeträge abenteuerliche:

- **80 Euro**

Na immerhin. Falls ihr den Client nutzt, er euch gefällt, oder ihr mich bei der Weiterentwicklung unterstützen möchtet, sind im Programm selbst und im User Manual die notwendigen Infos hinterlegt. Falls jemand etwas programmieren möchte, bin ich auch nicht abgeneigt.

Das Projekt / Source ist über GitHub öffentlich verfügbar (die Codebasis dort ist etwas älter, weil ich das mehrheitlich in einem lokalen git hoste).

Chat-Mining: Jagdinstinkte wecken mit KST4Contest

9) Download

Aktuell kann man den Client und das User Manual auf:

- <https://do5amf.funkerportal.de/>

Herunterladen.

Die neue Version 1.31 mit allen Features werde ich bis Mitte Februar fertigstellen und hochladen. Es stehen schon diverse OM an, die sie testen wollen. Es gibt aber eine Reihe von TODOs, die vorher noch erledigt werden müssen.

Da ich Ende 2024 eine Domain registriert habe, auf deren Server auch der Webservice für den QRG-Daten-Austausch laufen soll, werde ich auch den Downloadbereich dort hin verlagern. Die neue Adresse lautet dann:

- <https://hamradioonline.de/>

Vermutlich werde ich die Dateien noch eine ganze Weile lang auf beiden Domains hosten.

Bei Fragen um den Client bitte diese Mailadresse: praktimarc+kst4contest@gmail.com

73 und bis bald im Contest de DO5AMF // Marc @ DM5M